CS 147: Lab 01

February 3, 2026

The main objective of this problem set is to become familiar with using the toolchain for the course project. Feel free to refer to the Verilog cheatsheet, Verilog tutorial slides linked off of canvas.

Some advice: To deal with complexity, use a "divide and conquer" or hierarchical design approach. Divide the circuit into logical pieces, called blocks, which can be composed to form the larger circuit. For example, a 4-to-1 multiplexor or mux can be composed from 2-to-1 muxes. Hierarchical design reduces both the complexity faced by the designer and the complexity of the computer's representation of the schematic. While hierarchical design may seem unnecessary for something as simple as a 4-to-1 mux, remember that modern computers have millions of gates.

Testing Instructions

To run the testbench programs that we have distributed for this lab, use:

- `./run test <homework> [problem]`
- Replace `<homework>` with the assignment name (e.g., `hw1`).
- The `[problem]` argument is optional. If omitted, all tests for the selected homework assignment will be run.
- For more detailed output, add `-v` (e.g., `./run test -v hw1 hw1_1`).

Submission Instructions

Read the Verilog rules document in the assignments folder (see `Verilog Rules.pdf`) and adhere to the conventions.

To build a submission for grading, run:

- `./run build_submission <assignment>`

- This will automatically run `./run test <assignment>`, then run the generated submission through the integrated autograder.
- A submission ZIP is created in `generated_turnins/<assignment>/`.
- If you run `build_submission` multiple times for the same assignment, the generated zip files will be prepended with a number. The most recent submission should be the item with the highest number.
- IMPORTANT: Running `./run build_submission <assignment>` DOES NOT actively submit your assignment, it only generates the file you will submit. This file must be manually uploaded to Gradescope.

Problem 1

For all parts of this problem, we have provided templates that you can add your code to. Each problem is in its own directory (e.g., hw1_1).

1. Design a 1-bit 2-to-1 multiplexer module called mux2_1, using only `NAND`, `NOR`, and `NOT` gates. Implement the circuit in verilog using the provided basic gate modules. These modules are already included in the downloaded folders. Instantiate them in your module to use them. The input data lines of the multiplexer should be labeled *InA* and *InB*, the select line labeled *S*, and the output labeled *Out*. Your module should be named `mux2_1` and your file should be named `mux2_1.v`.
2. Use the 2-to-1 mux you designed in step 1 to hierarchically create a 4-to-1 mux called mux4_1. Label the inputs *InA, InB, InC*, and *InD*, and the output *Out*. Make your select input a 2-bit-wide bus (not a single wire); name it *S (1:0)*. (If *S* is `b00`, *InA* is selected; if *S* is `b01`, *InB* is selected, etc.)
3. Hierarchically create a quad 4-to-1 mux called quadmux4_1, using your 4-to-1 mux. The inputs to the new mux should be four 4-bit busses labeled *InA (3:0), InB (3:0), InC (3:0)*, and *InD (3:0)*. The select bus is labeled *S (1:0)* and the output should be a bus labeled *Out (3:0)*.
4. Use the testbench provided for testing.

Problem 2

For all parts of this problem, we have provided templates that you can add your code to. I also recommend consulting Appendix B.5-B.6 in your textbook if you need help getting started. You may also find B.3 (Combinational Logic) and B.4 (Using HDLs) useful. Similar rules apply throughout this problem – you should build your larger combinational logic out of the provided `NOT, NAND, NOR`, and `XOR` gates. If you need an assign statement to connect

a single bit (e.g., assign c[0] = c_in;) that is ok, but larger logic should be using the provided gates.

1. Design a 1-bit full adder using only the provided `NOT, NAND, NOR`, and `XOR` gates (you do not have to use all four types of gates, you are just limited to using these gate types). Again use the provided modules. Label the inputs as *inA*, *inB*, and *cIn* (carry-in). Label the outputs as *s* and *cOut*. Note that this will look similar to the adder we designed in class. Your module should be named `fullAdder1b` and your file should be named `fullAdder1b.v`.

2. Optionally (but strongly recommended), verify the correctness of the 1-bit adder over all combinations of inputs by writing your own testbench. It is generally best to test at the smallest module level first. Given that this module is so small, it should be fairly easy to exhaustively test it. I also recommend that you create "self-checking" testbenches that compare the answer of the module you designed to the "golden" output that you get from using "+" or "- " in the testbench (again, it is ok to use these in the testbench, but not in the modules you are designing). See the testbench in Step 5 (below) for an example of this.

3. Using the 1-bit full adder you created above, design a carry lookahead adder (CLA) with the same gate restrictions that adds two 4-bit binary numbers. Make the inputs and outputs 4-bit buses (vectors) labeled *inA(3:0)*, *inB(3:0)*, and *sum(3:0)*, respectively. Label the carry-in *cIn* and the carry-out *cOut*. Your module should be named `cla4b` and your filename should be `cla4b.v`.

4. Using the 4-bit CLA you created above, design a CLA that adds two 16-bit binary numbers with the same gate restrictions. Make the inputs and outputs 16-bit buses (vectors) labeled *inA(15:0)*, *inB(15:0)*, and *sum(15:0)*, respectively. Label the carry-out from the adder *cOut* and the carry-in *cIn*. Your module should be named `cla16b` and your file should be named `cla16b.v`.

   (a) Note: you may add P and G inputs/outputs to your `cla4b` module for this.

5. Use the testbench provided for testing.